

Machine Learning in Economics

Dacha Hitesh Kumar

Abstract

Economists use traditional and long complex methods to find the current economic status of a specific country. Machine learning helps in finding this data in a quite similar way but with less complexity and more accuracy. Here I will be developing a model that will be trained on the economic data of African countries and it will predict which countries will face the crises based on data.

Keywords: Economic Crisis, Machine Learning, Economic growth prediction, Bankruptcy, economic crisis prediction

Indexes

- i. Introduction
- ii. ML in Economics
- iii. Data Preprocessing and feature Engineering
- iv. Model Building
- v. Conclusion
- vi. Reference

i. Introduction

In today's world Machine Learning and Artificial Intelligence have become an integral part of our day-to-day lives, as some organizations have begun using AI and Machine Learning which supports them in developing new products. Machine Learning is algorithms that are trained in a way that it learns how people react in certain situations by itself.

Like all other industries, the economy, and finance industries use machine learning and artificial intelligence algorithm to get the solution or know some unknown events in advance which be handled in a planned manner.

Economists use complex formulas and big theorems to prove and find the economic situation of anything or a country. Using machine learning this process becomes faster and more efficient but the application needs to be defined carefully with appropriate data as the field itself is complex to understand. Economic crises, financial stability, and economic growth are some of the examples where machine learning can be used.

ii. ML in Economics

In my project, I am using a dataset that contains data related to the financial stability of African Countries. The dataset contains data about banking, debts, inflation, systematic crisis, and the financial state of different countries from the period 1860 to 2014. This dataset is a subset of Reinhart et. AL's Global Financial Stability dataset on this dataset.

It's a huge dataset containing data from different regions of Africa. I will use different techniques to visualize the data and analyze it to make that data more sensible for the training model on this data.

I will apply different classification models like decision trees, Random Forest, Gradient Boosting, Support Vector Machine, K Nearest Neighbor, and Gaussian Naive Bias to train the model and predict whether the given country will face a crisis or not. This will help to handle the situation that is going to arrive in the future in a better way by planning the economy and alerting citizens on certain issues in advance.

iii. Data Preprocessing and feature Engineering

Machine Learning will help us to find patterns out of the data which will be for prediction and to get this prediction correct, data needs to be construction in the correct format and the process of constructing this data is known as data preprocessing and feature engineering will help to scale and transform the data model training.

In my case study, I will use different preprocessing and visualization techniques to make raw data in a proper format.

Before starting preprocessing steps, I will import all the functions and libraries that will be required during preprocessing and visualization steps and in the modelling step.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go

from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
```

Figure 1

Now I will read the data from the CSV file into a data frame and display using the head function.

```
df_cur = pd.read_csv('african_crises.csv')
df_cur.head()
```

	case	cc3	country	year	systemic_crisis	exch_usd	domestic_debt_in_default	sovereign_external_debt_default	gdp_weighted_default	inflation_annual_cpi
0	1	DZA	Algeria	1870	1	0.052264	0	0	0.0	3.441456
1	1	DZA	Algeria	1871	0	0.052798	0	0	0.0	14.149140
2	1	DZA	Algeria	1872	0	0.052274	0	0	0.0	-3.718593
3	1	DZA	Algeria	1873	0	0.051680	0	0	0.0	11.203897
4	1	DZA	Algeria	1874	0	0.051308	0	0	0.0	-3.848561

Figure 2

Now I will check what the data looks like in terms of datatype and how many records and features in total are there and what is the size of the total dataset.

```
df_cur.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1059 entries, 0 to 1058
Data columns (total 14 columns):
 #   Column                                Non-Null Count  Dtype  
---  -
 0   case                                1059 non-null  int64  
 1   cc3                                1059 non-null  object  
 2   country                             1059 non-null  object  
 3   year                                1059 non-null  int64  
 4   systemic_crisis                     1059 non-null  int64  
 5   exch_usd                            1059 non-null  float64 
 6   domestic_debt_in_default            1059 non-null  int64  
 7   sovereign_external_debt_default     1059 non-null  int64  
 8   gdp_weighted_default                1059 non-null  float64 
 9   inflation_annual_cpi                1059 non-null  float64 
10   independence                        1059 non-null  int64  
11   currency_crises                     1059 non-null  int64  
12   inflation_crises                     1059 non-null  int64  
13   banking_crisis                      1059 non-null  object  
dtypes: float64(3), int64(8), object(3)
memory usage: 116.0+ KB
```

Figure 3

Now our target variable is in the text format and hence convert this text into a numeric format using a label encoder and then visualize the data again using the head function.

```
df_cur.loc[:, 'banking_crisis'] = -(LabelEncoder().fit_transform(df_cur.banking_crisis)-1)
df_cur.banking_crisis.unique()
array([1, 0], dtype=object)
```

Figure 4

```
df_cur.head()
```

	case	cc3	country	year	systemic_crisis	exch_usd	domestic_debt_in_default	sovereign_external_debt_default	gdp_weighted_default	inflation_annual_cpi
0	1	DZA	Algeria	1870	1	0.052264	0	0	0.0	3.441456
1	1	DZA	Algeria	1871	0	0.052798	0	0	0.0	14.149140
2	1	DZA	Algeria	1872	0	0.052274	0	0	0.0	-3.718593
3	1	DZA	Algeria	1873	0	0.051680	0	0	0.0	11.203897
4	1	DZA	Algeria	1874	0	0.051308	0	0	0.0	-3.848561

Figure 5

Now I will check null values present in each feature using is null function.

```
df_cur.isnull().sum()
```

case	0
cc3	0
country	0
year	0
systemic_crisis	0
exch_usd	0
domestic_debt_in_default	0
sovereign_external_debt_default	0
gdp_weighted_default	0
inflation_annual_cpi	0
independence	0
currency_crises	0
inflation_crises	0
banking_crisis	0
dtype: int64	

Figure 6

Now I will try to see how the data distribution is done in terms of mean and standard deviation.

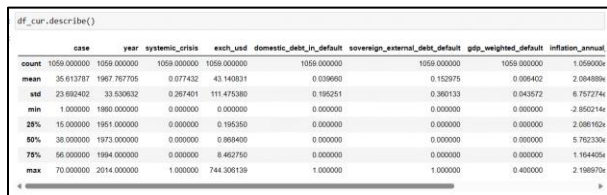


Figure 7

Now let see how the data is distributed with respect to target label values and I can see value 0 data is quite high compare to value 1 as a target.

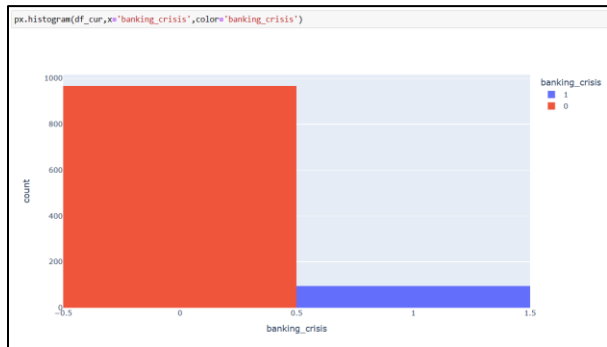


Figure 8

I will plot the line graph which shows the country wise annual inflation rate.



Figure 9

I will plot bar graph which shows the inflation rate for every country over the years.



Figure 10

Now I will prepare a few more data frames that will be used for data visualization as well as data modeling.

```
df_cur.currency_crises.replace(2,1,inplace=True)
df_cur.sort_values(['country','year'],inplace=True)
Location = df_cur[['case','cc3','country','year']].copy()
data_Info = df_cur.drop(columns=Location.columns.values.tolist()).copy()
```

Figure 11

Now I will plot the data on the map which shows different crises over the year for specific countries.

First, I will plot systemic crisis on the map.

```
df_cur_new = df_cur.copy()
df_cur_new.country.replace('Egypt','AaEgypt',inplace=True)
df_cur_new.sort_values(['country','year'],inplace=True)

my_color_scale = [ [0,'blue'],[1,'red'] ]

fig = px.choropleth(data_frame=df_cur_new,locations='country',locationmode='country names',
                    color='systemic_crisis',
                    color_continuous_scale=my_color_scale,
                    range_color=[0,1],
                    hover_name='country',
                    hover_data=['cc3','systemic_crisis','currency_crises','inflation_crises',
                                'banking_crisis'],
                    scope='africa',animation_frame='year',projection='natural earth')

fig.update_layout(title='Systemic Crisis by Year',
                  coloraxis_colorbar=dict(ticks='outside',tickvals=[0,1],
                                          title='Systemic Crisis',
                                          ticktext=['No','Yes']))

fig.show()

print('Gray countries are those whose information is not available in the dataset')
```

Figure 12

Figure 13 shows the map graph with a slider to move from one year to another and see which country was facing a systemic crisis in a particular year.

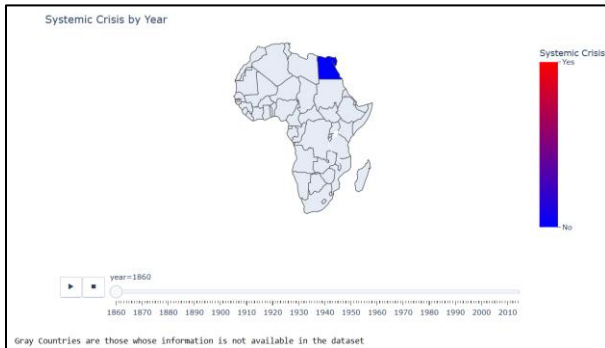


Figure 13

The next plot map is for the currency crisis. Figure 15 countries that faced currency crises corresponding can be seen using the slider.



Figure 14

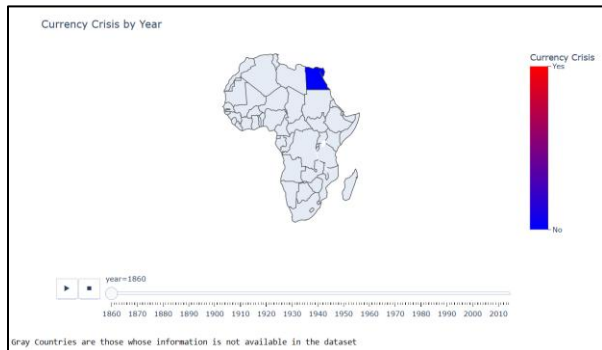


Figure 15

The next plot is for the inflation crisis over the year. Again, using the slider, a country that faced an inflation crisis can be visualized over the year.



Figure 16

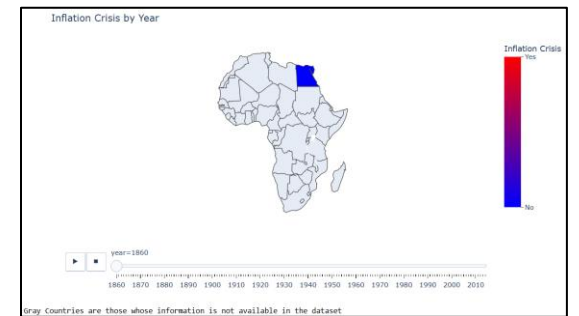


Figure 17

Banking Crisis is the next plot that is plotted for different countries over the year which can be visualized in Figure 19.

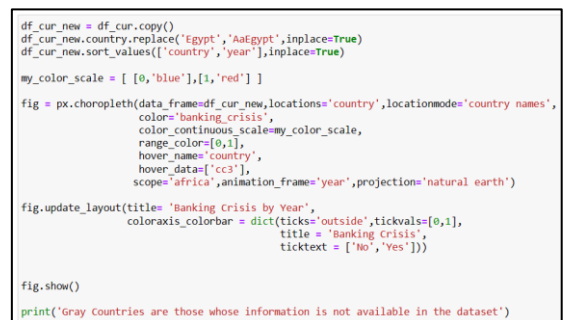


Figure 18

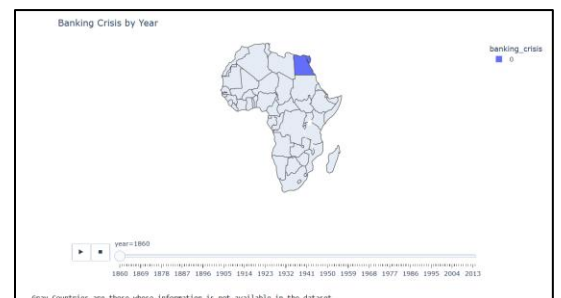


Figure 19

The next plot demonstrates Exchange to US dollars by year for a different country.

```
df_cur_new = df_cur.copy()
df_cur_new.country.replace('Egypt', 'AaEgypt', inplace=True)
df_cur_new.sort_values(['country', 'year'], inplace=True)

fig = px.choropleth(data_frame=df_cur_new, locations='country', locationmode='country names',
                    color='exch_usd',
                    color_continuous_scale='viridis',

                    hover_name='country',
                    hover_data=['cc3', 'systemic_crisis', 'currency_crises', 'inflation_crises',
                                'banking_crisis'],
                    scope='africa', animation_frame='year', projection='natural earth')

fig.update_layout(title='Exchange to US Dollars by Year',
                  coloraxis_colorbar=dict(ticks='outside',
                                          title='Exchange',
                                          ))

fig.show()
print('Gray Countries are those whose information is not available in the dataset')
```

Figure 20

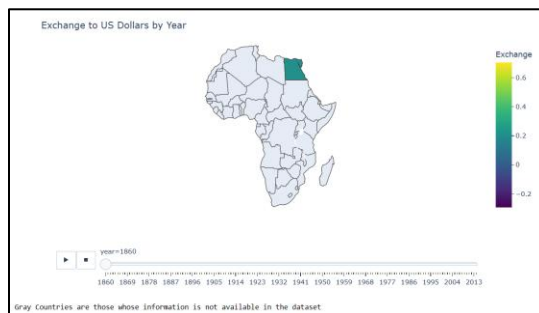


Figure 21

Annual inflation by year is the map plot that I have plotted as visualized in Figure 23

```
df_cur_new = df_cur.copy()
df_cur_new.country.replace('Egypt', 'AaEgypt', inplace=True)
df_cur_new.sort_values(['country', 'year'], inplace=True)

fig = px.choropleth(data_frame=df_cur_new, locations='country', locationmode='country names',
                    color='inflation_annual_cpi',
                    color_continuous_scale='viridis',

                    hover_name='country',
                    hover_data=['cc3', 'systemic_crisis', 'currency_crises', 'inflation_crises',
                                'banking_crisis'],
                    scope='africa', animation_frame='year', projection='natural earth')

fig.update_layout(title='Annual Inflation by Year',
                  coloraxis_colorbar=dict(ticks='outside',
                                          title='Annual Inflation',
                                          ))

fig.show()
print('Gray Countries are those whose information is not available in the dataset')
```

Figure 22

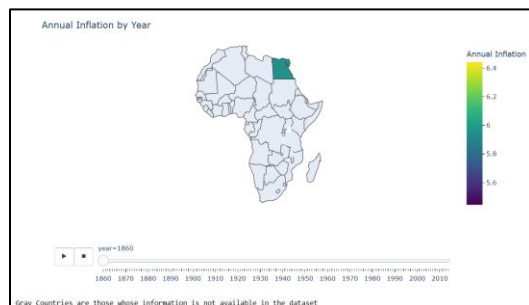


Figure 23

After visualizing the different data on the map let's find the co-relation between different features and for that, I have plotted the heatmap for this data.

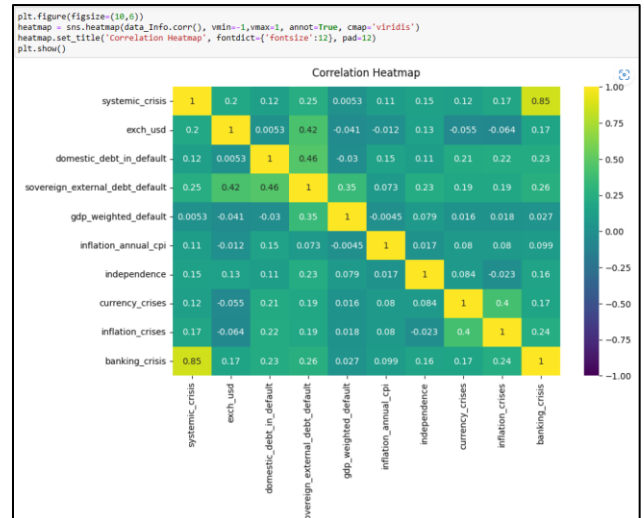


Figure 24

Figure 24 shows mostly all the features independent apart from the banking crisis and systemic crisis, these two features have high co-relation.

So now I have done enough analysis on the data and converted the data in the required format which can be used for model training and hence the next thing that I will do is the model training.

iv. Model Building

For training the model the data needs to be splitted and before that target variable and other features need to be separated.

```
target_var = data_Info.systemic_crisis
pred_var = data_Info.drop(columns='systemic_crisis')
X_train, X_test, Y_train, Y_test = train_test_split(pred_var, target_var)
```

Figure 25

Now I start training for random forest classifier and test it with testing data to find the accuracy score and error which is 96% and 3.396% respectively. Also, to get the better understanding I printed the confusion matrix.

```
Random_Forest = RandomForestClassifier()
Random_Forest.fit(X_train,Y_train)
print("Accuracy score : {}".format(Random_Forest.score(X_test,Y_test)))
Y_pred = Random_Forest.predict(X_test)

confusion_mat = confusion_matrix(Y_test,Y_pred)

sns.set_style(style='dark')
plt.figure(figsize=(8,6))
conf_mat = sns.heatmap(confusion_mat, vmin=np.min(confusion_mat.all()),
vmax=np.max(confusion_mat), annot=True,fmt='d', annot_kws={"fontsize":15})
conf_mat.set_title('Confusion Matrix', fontdict={'fontsize':12}, pad=12)
error = (1-np.diag(confusion_mat).sum()/len(Y_test))*100
print('Error is : {}'.format(error))

plt.show()
```

Figure 26

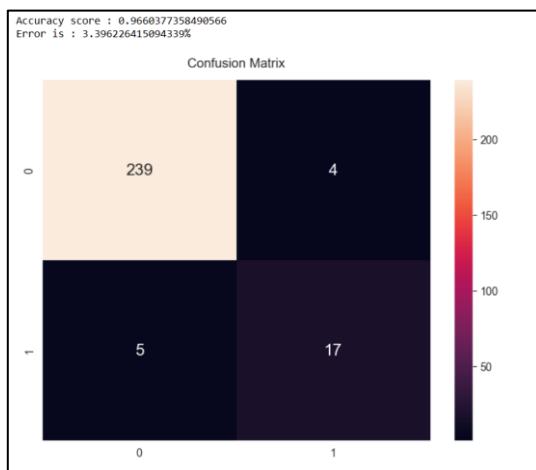


Figure 27

The next model that I will train is the decision tree classifier and after testing the trained model I got an accuracy of 95.84% with an error rate of 4.15%.

```
Decision_Tree = DecisionTreeClassifier()
Decision_Tree.fit(X_train,Y_train)
print("Accuracy score : {}".format(Decision_Tree.score(X_test,Y_test)))
Y_pred = Decision_Tree.predict(X_test)

confusion_mat = confusion_matrix(Y_test,Y_pred)

sns.set_style(style='dark')
plt.figure(figsize=(8,6))
conf_mat = sns.heatmap(confusion_mat, vmin=np.min(confusion_mat.all()),
vmax=np.max(confusion_mat), annot=True,fmt='d', annot_kws={"fontsize":15})
conf_mat.set_title('Confusion Matrix', fontdict={'fontsize':12}, pad=12)

error = (1-np.diag(confusion_mat).sum()/len(Y_test))*100
print('Error is : {}'.format(error))

plt.show()
```

Figure 28

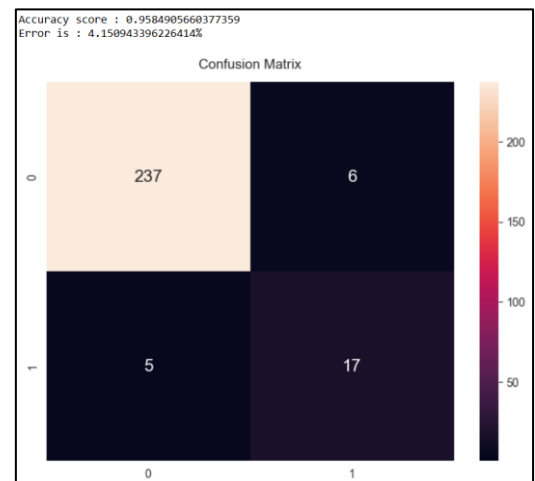


Figure 29

Gradient Boosting Classifier is the next model that I will train and test and after testing accuracy that I got is 95.84% and 4.15% of error which is the same as the previous decision tree model.

```
GBC = GradientBoostingClassifier()
GBC.fit(X_train,Y_train)
print("Accuracy score : {}".format(GBC.score(X_test,Y_test)))
Y_pred = GBC.predict(X_test)

confusion_mat = confusion_matrix(Y_test,Y_pred)

sns.set_style(style='dark')
plt.figure(figsize=(8,6))
conf_mat = sns.heatmap(confusion_mat, vmin=np.min(confusion_mat.all()),
vmax=np.max(confusion_mat), annot=True,fmt='d', annot_kws={"fontsize":15})
conf_mat.set_title('Confusion Matrix', fontdict={'fontsize':12}, pad=12)

error = (1-np.diag(confusion_mat).sum()/len(Y_test))*100
print('Error is : {}'.format(error))

plt.show()
```

Figure 30

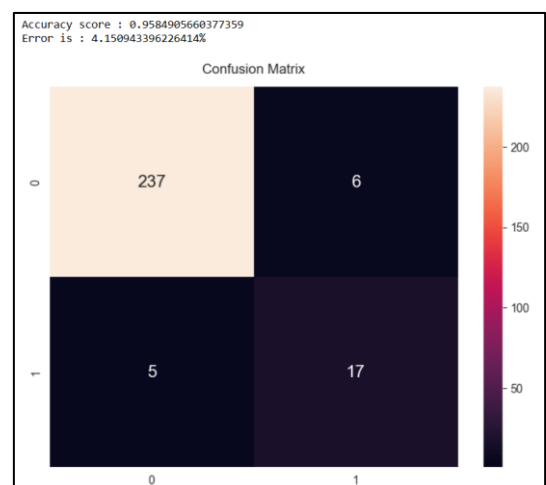


Figure 31

Now I will train the Gaussian Naïve Bias model on the training dataset and test to find the accuracy which is 92.45% and the error comes out to be 7.54%.

```
Gaussian_NB = GaussianNB()
Gaussian_NB.fit(X_train,Y_train)
print('Accuracy score : {}'.format(Gaussian_NB.score(X_test,Y_test)))

Y_pred = Gaussian_NB.predict(X_test)

confusion_mat = confusion_matrix(Y_test,Y_pred)

sns.set_style(style='dark')
plt.figure(figsize=(8,6))
conf_mat = sns.heatmap(confusion_mat, vmin=np.min(confusion_mat.all()),
                        vmax=np.max(confusion_mat), annot=True,fmt='d', annot_kws={"fontsize":15})
conf_mat.set_title('Confusion Matrix', fontdict={'fontsize':12}, pad=12)

error = (1-np.diag(confusion_mat).sum()/len(Y_test))*100
print('Error is : {}'.format(error))

plt.show()
```

Figure 32

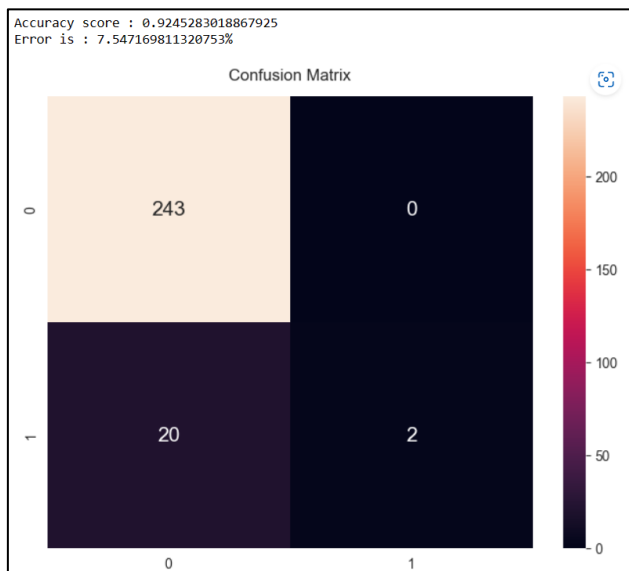


Figure 33

With k vale as 5, I will train the KNN model now and test to predict the output and find the accuracy and error. The accuracy percentage of the KNN model is 90.94% and 9.05% is the error rate.

```
KNN = KNeighborsClassifier(n_neighbors=5)
KNN.fit(X_train,Y_train)
print('Accuracy score : {}'.format(KNN.score(X_test,Y_test)))

Y_pred = KNN.predict(X_test)

confusion_mat = confusion_matrix(Y_test,Y_pred)

sns.set_style(style='dark')
plt.figure(figsize=(8,6))
conf_mat = sns.heatmap(confusion_mat, vmin=np.min(confusion_mat.all()),
                        vmax=np.max(confusion_mat), annot=True,fmt='d', annot_kws={"fontsize":15})
conf_mat.set_title('Confusion Matrix', fontdict={'fontsize':12}, pad=12)

error = (1-np.diag(confusion_mat).sum()/len(Y_test))*100
print('Error is : {}'.format(error))

plt.show()
```

Figure 34

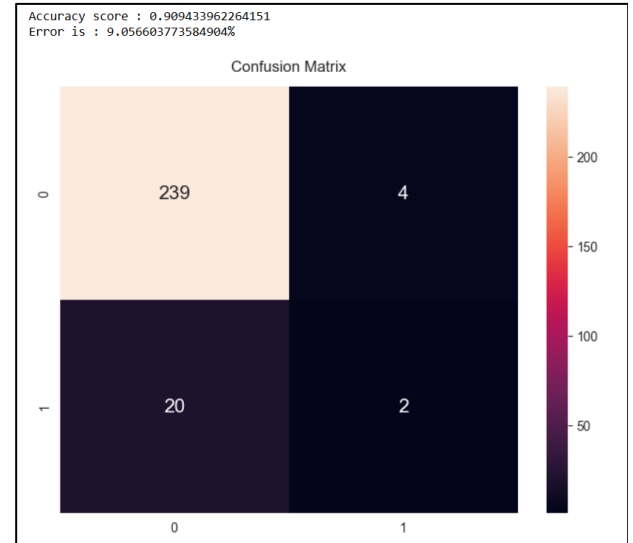


Figure 35

The last model that will train is the support vector machine and after testing on the test dataset accuracy that I got is 91.69% with an 8.30% error.

```
SVM = SVC(probability=True)
SVM.fit(X_train,Y_train)
print('Accuracy score : {}'.format(SVM.score(X_test,Y_test)))

Y_pred = SVM.predict(X_test)

confusion_mat = confusion_matrix(Y_test,Y_pred)

sns.set_style(style='dark')
plt.figure(figsize=(8,6))
conf_mat = sns.heatmap(confusion_mat, vmin=np.min(confusion_mat.all()),
                        vmax=np.max(confusion_mat), annot=True,fmt='d', annot_kws={"fontsize":15})
conf_mat.set_title('Confusion Matrix', fontdict={'fontsize':12}, pad=12)

error = (1-np.diag(confusion_mat).sum()/len(Y_test))*100
print('Error is : {}'.format(error))

plt.show()
```

Figure 36

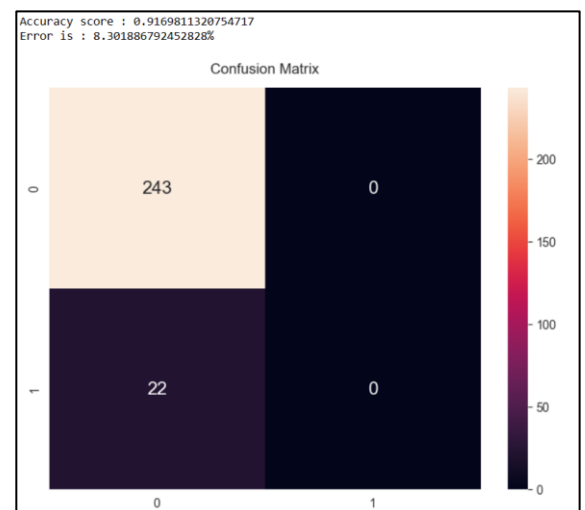


Figure 37

Hence the best model out of all is Random Forest Classifier and I will run the model again on the test data set and compare the predicted output with the actual output to see how well the model works.

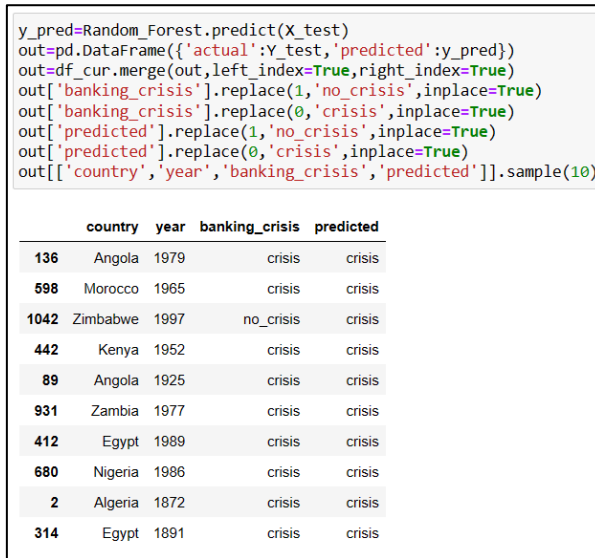


Figure 38

v. Conclusion

Here in my project, I have trained 6 different models on the dataset that shows the financial status of the countries in Africa between 1870 to 2014. All the models that were trained have given the output accuracy of more than 90% but out of all the random forest classifiers is the model that is given the best accuracy of 96.60% and minimal error rate of 3.39% and hence this model can be used to predict the financial condition of any country.

vi. References

- [1] Huang Junyu: Prediction of Financial Crisis Based on Machine Learning
- [2] Aristeidis Samitas, Elias Kampouris, Dimitris Kenourgios: Machine learning as an early warning system to predict a financial crisis
- [3] Wei-Yang Lin, Ya-Han Hu, Chih-Fong Tsai: Machine Learning in Financial Crisis Prediction: A Survey
- [4] Blog by Sebastian Petric, CFA: Can Machine Learning help predict the Next Financial Crisis? <https://blogs.cfainstitute.org/investor/2022/10/20/can-machine-learning-help-predict-the-next-financial-crisis/>
- [5] Data Visualization on map official documentation : <https://plotly.com/python/choropleth-maps/>
- [6] Line plotting official documentation : [https://plotly.com/python/plotly-express/#:~:text=The%20plotly.express%20module%20\(usually,for%20creating%20most%20common%20figures.](https://plotly.com/python/plotly-express/#:~:text=The%20plotly.express%20module%20(usually,for%20creating%20most%20common%20figures.)
- [7] Anuganti Suresh : What is Confusion Matrix : <https://medium.com/analytics-vidhya/what-is-a-confusion-matrix-d1c0f8feda5>
- [8] Official documentation for model training : <https://scikit-learn.org/stable/>
- [9] Zach : How to perform Label Encoding in Python(With Example) : <https://www.statology.org/label-encoding-in-python/>